

SGGotWhat? - Team02

SGGotWhat? - Team02 (*student proposal*)

Assessment: ♦ Fit with adjustments · **Category:** Social Media · **Assessed:** 2026-09-21 · **Status:** awaiting instructor approval

Proposal (as submitted): A crowdsourced discovery app helping Singaporeans find real hidden gems (quiet cafes, unexpected viewpoints), countering the "SG got what" stereotype. Users log spots and one-off events, validated by the community; a personalized recommendation engine and route planner surface what's worth seeing and plan a full day out. An Events page pairs Eventfinda-API listings with community-submitted niche pop-ups. Backend C++ services handle spatiotemporal search, trending and multi-stop transit route optimization in real time. Named technical features: a **Trust & Validation pipeline** (hard checks — GPS radius + accuracy, rate limiting, account age; soft checks — perceptual photo hashing, n-gram/Jaccard text similarity); a **personalised recommendation engine** (per-user affinity vectors + collaborative persona matching, a reserved share from adjacent clusters to avoid taste bubbles, clusters recomputed in a background loop); a **spatiotemporal discovery index** (custom C++ spatial index over spots/events with valid-time windows, distance blended with trending/ personalisation, auto-expanding radius, zoom-based pin clustering); and a **multimodal router + itinerary optimiser** (C++ earliest-arrival over bus/MRT from LTA DataMall, plus a time-window orienteering metaheuristic that selects/orders stops to maximise value). Custom tool: a web-based moderation & content editor (review soft-check-flagged submissions side-by-side with the detected duplicate, create/edit spots & categories, tune ranking weights with a live preview of a test user's feed). Perf targets: nearby query <50 ms p95 at 10k spots; itinerary <2 s for 20 candidates; submission validation <500 ms; deterministic-for-seed recommendations.

Adjustments required for approval:

- **Clear the 2D gate with a custom-rendered data-display surface.** A map built purely from a maps SDK plus native lists would miss the module's 2D

requirement. Custom-render at least the map layer (your own pin-clustering, auto-expanding-radius ring, route polyline and trending/popularity styling) and/or the day-out itinerary visualization in **OpenGL ES (GLSurfaceView)** — the editor's live "feed re-sort" preview is another good candidate — rather than only a drop-in map widget and RecyclerViews.

- **Safety & privacy for user-submitted spots and photos.** Keep the trust/validation pipeline as a graded surface and add report/block plus moderation of user photos before public listing; don't expose a submitter's precise identity/home or a private draft location. (Your hard/soft checks and moderation tool already cover most of this — make the user-facing safety controls explicit.)

*(Not required, affirmed: LTA DataMall and the Eventfinda API are legitimately available data sources — good choice over scraping. Keep the spatial index, recommender, router and orienteering solver as the team's own load-bearing C++.)**

References:

- [Atlas Obscura — Wikipedia](#) (crowdsourced catalogue of unusual/hidden places via user-generated + editorial content, plus local community experiences and events)
- [Foursquare City Guide — Wikipedia](#) (local search-and-discovery with personalized recommendations from history, a community-edited venue database and user trust levels)
- [Citymapper — Wikipedia](#) (multimodal public-transit journey planning with live timings across walk/bus/metro, built on open GTFS transit data)

Core Tech Requirements

Legend: ✓ Applies directly | ♦ Applies with domain adaptation | ● Optional | ✕ Not needed | » Second trimester

Core Tech Requirement	SGGotWhat?
External Database	✓
Authentication »	✓
Analytics »	✓
Real-time C++ Simulation Loop and/or C++ Performant Service	✓
Application Communication	✓
Continuous Integration	✓
Data-Driven Architecture	✓
Front-end + Back-end	✓
Custom Tools Development	✓
2D Graphical Output	♦
3D (strong students only)	✕
Networking System	✓

How each requirement applies:

Platform fit: **Main:** Native mobile + Web browser on mobile — a location-first discovery app (map, nearby, on-the-go itineraries) on the phone (device: Pixel 9a); the web-based moderation/content editor and the C++ desktop benchmark build are the tools/dev vehicle. **Extended:** Firebase + Linux server set up by M2 (full use T2).

Front end: The map (custom pin-clustering, auto-expanding radius, route polyline, popularity styling), the personalised discovery feed, the events browse/filter page, the day-out itinerary view and member profiles (OpenGL

ES for the custom map/itinerary surface). **Languages:** Web — HTML/JS/CSS + C++→WASM (Emscripten, SDL2/3, OpenGL ES) for the moderation tool & desktop benchmark build; Native — Kotlin Native Android UI + C++ via JNI (Android Studio); OpenGL ES in a GLSurfaceView where realtime 2D is needed (no SDL on native).

Back end: The C++ services — the spatiotemporal spatial index (distance × trending × personalisation), the recommender (affinity vectors + persona clustering, background recompute), the trust/validation pipeline (GPS/hash/text-sim), and the multimodal transit router + time-window orienteering solver (LTA DataMall) — plus a Firebase store for spots/events/profiles and an Eventfinda-API adapter. **Languages:** C++.

Backend scope: Local now, server-ready (T2) — Firebase for the crowdsourced DB; the Linux server seam set up & basic-tested by M2, full use in T2.

- *Loop and/or service:* several load-bearing **C++ services** — an in-memory spatial index answering nearby queries under 50 ms p95 at 10k spots, an orienteering solver returning a ≤20-stop itinerary in under 2 s, a validation pass under 500 ms per submission, and a fixed-interval background loop recomputing persona clusters and trending without blocking requests (deterministic for a given seed, so the recommender is reproducibly testable).
- *Data-driven:* spots/events (with valid-time windows), categories, affinity/persona vectors, ranking weights (distance/trending/personalisation/exploration) and validation thresholds are hot-reloadable JSON — the editor retunes ranking without recompiling.
- *2D:* the map (own clustering/radius/route/popularity rendering) and the itinerary/feed views are **custom-rendered in OpenGL ES** (see adjustment 1). *3D:* not needed. *Networking:* Eventfinda + LTA DataMall API calls, Firebase sync of the crowdsourced DB, and push for trending/events.
- *Custom tool:* the **web-based moderation & content editor** — review soft-check-flagged submissions beside the detected duplicate, create/edit spots & categories, and tune ranking weights with a live preview of a test user's feed.
- *External DB:* Firebase for spots/events/profiles/trust. *Auth* »: accounts + trust level (account age is a validation input). *Analytics* »: engagement events feed trending and persona clustering.

Suggested Tech Goals

G1–G3 ★ are the common goals (fixed for all categories); the other 3 are picked from the Social Media pool in [milestone-goals-pool-per-category.pdf](#) (G1–G3 ★ common, G4+ picks).

M1:

- **G1 ★ Application Shell, Real-time C++ Simulation Loop, Input & Core Logic**

Tech & dependencies

- **Tech: Web build:** On the desktop/benchmark build, C++17/20 with a service loop + a fixed-interval background pass (trending/persona recompute); SDL2/3 dev harness, CMake, Git. **Native build:** Android Studio (Kotlin Native Android UI + the C++ engine via JNI); OpenGL ES drawn in a GLSurfaceView where realtime 2D is needed — no SDL on native, some screens are pure Native UI.
- **Prerequisites:** None — foundation of this project.
- **Feeds into:** Every other goal; the M1G9 recommender and the background loop run through it.

- **G2 ★ Data-Driven Architecture of Core Objects / Entity System**

Tech & dependencies

- **Tech:** C++ entity/registry with spots/events (valid-time windows), categories, affinity/persona vectors, ranking weights and validation thresholds in JSON (nlohmann/json) with hot-reload.
- **Prerequisites:** G1 (loop & update hooks).
- **Feeds into:** M1G9 (scoring reads it), M2G2 (the editor tunes it), M2G11 (the spatial/discovery index), M1G13 (validation).

- **G3 ★ Debugging, Profiling, Stress Test & CI**

Tech & dependencies

- **Tech:** ImGui overlays for the ranker/index, Tracy profiling against the perf targets (nearby <50 ms p95 @10k, itinerary <2 s, validation <500 ms), Catch2 tests incl. seeded-deterministic recommender checks, GitHub Actions Windows+Linux matrix.
- **Prerequisites:** G1 (loop to instrument).
- **Feeds into:** M2G3 (CI hardening); the perf budgets M2G1 validates on-device.

- **G9 Matching / Ranking Engine — Foundation** — the recommender: blend distance, trending and per-user affinity/persona into ranked results.

Tech & dependencies

- **Tech:** Weighted scoring over affinity vectors + persona match, a reserved exploration share from adjacent clusters, and incremental re-ranking (custom C++) — not a database query.
- **Prerequisites:** G2 (spot/user model), G1 (tick + background recompute).
- **Feeds into:** M2G8 (recommender II + clustering), M2G11 (spatiotemporal index), the feed.

- **G6 Custom 2D UI Rendering** — the map (custom pin-clustering, auto-expanding radius, route, popularity) rendered custom (satisfies the 2D gate).

Tech & dependencies

- **Tech: Web build:** SDL2/3 renderer or OpenGL ES, GLSL, FreeType, stb_image for custom markers, cluster bubbles, the radius ring and route polyline on the desktop build. **Native build:** Android Studio (Kotlin Native Android UI + the C++ engine via JNI); OpenGL ES drawn in a GLSurfaceView where realtime 2D is needed — no SDL on native, some screens are pure Native UI.
- **Prerequisites:** G1 (shell & loop).
- **Feeds into:** The nearby map + itinerary views; the editor's live preview.

- **G13 Moderation Data Model & Report Flow — Foundation** — the Trust & Validation pipeline (hard + soft checks) and the report/flag flow.

Tech & dependencies

- **Tech:** Hard checks (GPS radius + accuracy, rate limiting, account age) and soft checks (perceptual photo hashing, n-gram/Jaccard text similarity) in C++ under 500 ms; a report/flag queue in Firestore.
- **Prerequisites:** G2 (spot model); Firebase identity.
- **Feeds into:** M2G6 (the moderation dashboard/editor); trustworthy data for ranking.

M2:

- **G1 ★ Architecture — Optimization & Platform Validation**

Tech & dependencies

- **Tech: Web build:** Tracy profiling against the perf targets, ASan on MSVC, memory pools for the index, and desktop benchmark

validation; Linux server set up (account + software + basic connectivity test) by M2. **Native build:** Android Studio (Kotlin Native Android UI + the C++ engine via JNI); OpenGL ES drawn in a GLSurfaceView where realtime 2D is needed — no SDL on native, some screens are pure Native UI; validate nearby-query and itinerary latency and the GLSurfaceView map on the Pixel 9a.

- **Prerequisites:** All M1 goals (hardens the M1 codebase).
- **Feeds into:** The native deliverable; the p95/latency targets met on-device.

- **G2 ★ Content Editor — Core** — the web-based moderation & content editor: review flagged submissions beside duplicates, create/edit spots & categories, tune ranking weights with a live feed preview.

Tech & dependencies

- **Tech:** C++ + Dear ImGui (or a web front-end), JSON serialization, undo/redo; a live binding that re-sorts a test user's feed as weights change, and a side-by-side duplicate-review pane.
- **Libraries/Software:** Dear ImGui, nlohmann/json, the project's SDL2/3/OpenGL ES renderer (live feed/map preview).
- **Prerequisites:** G2 (data model), M1G9 (engine to preview), M1G13 (flagged submissions).
- **Feeds into:** Trusted content; ranking-weight tuning M2G8 consumes.

- **G3 ★ CI — CMake, Code Quality, Build Standards & Packaging**

Tech & dependencies

- **Tech: Web build:** CMake presets, clang-format + clang-tidy gates, and packaging of the moderation tool / desktop benchmark build via GitHub Actions. **Native build:** Android Studio (Kotlin Native Android UI + the C++ engine via JNI) packaging the Android APK/AAB; OpenGL ES in a GLSurfaceView where realtime 2D is needed — no SDL on native.
- **Prerequisites:** G3 (CI and test foundation).
- **Feeds into:** Release/showcase builds of every SGGotWhat? goal.

- **G11 Search & Discovery Engine** — the spatiotemporal discovery index: search spots/events by location + time window with auto-expanding radius.

Tech & dependencies

- **Tech:** A custom C++ spatial index (grid/quadtree) over spots and events with valid-time windows, distance blended with trending/

personalisation, auto-expanding radius and zoom-based pin clustering.

- **Prerequisites:** M1G2 (content model), M1G9 (scoring).
- **Feeds into:** The nearby map, the events page and the itinerary candidate set.

- **G8 Ranking / Matching Engine II** — recommendations II: persona clustering with an exploration share, refreshed by the background loop.

Tech & dependencies

- **Tech:** Collaborative persona clustering, similarity scoring, an adjacent-cluster exploration ratio (anti-taste-bubble), evaluation metrics, and a fixed-interval background recompute that never blocks requests.
- **Prerequisites:** M1G9 (ranking foundation).
- **Feeds into:** The personalised feed; the itinerary value model.

- **G7 Firebase — Auth, Push & Analytics** — accounts + trust level, the crowdsourced DB, trending/events push, and analytics.

Tech & dependencies

- **Tech:** Firebase Auth (accounts; account age feeds validation), Firestore for spots/events/profiles/trust, Cloud Messaging for trending/event push, Analytics — REST API from C++, JS/Kotlin SDK on the front-end.
- **Prerequisites:** M1G2 (data model), M1G13 (validation/trust).
- **Feeds into:** Member profiles/trust level; trending signals for ranking.

(Proposed new pool goal (requires instructor approval): **Multimodal Transit**

Router & Time-Window Orienteering Solver — earliest-arrival routing over bus/MRT from LTA DataMall (frequency-based estimates) plus a fixed-budget orienteering metaheuristic that selects and orders stops to maximise value; the Social Media pool has no routing/optimiser goal, so this would be a new appended pool goal. The spatiotemporal index above is the spatial variant of G11; the three picks stand regardless.)

Track Goals & Team Composition

Team composition (set by instructor): 6 CS + 1 Design + 0 Art (7) — SGGotWhat? is a UX-heavy social utility (map/nearby discovery, the personalised feed, events filtering, day-out itineraries, profiles/trust and the community-submission flow), and the visual surface (map markers, category icons, trust badges) is UI-art the designer owns while photos are user-supplied — no character/illustration production, so 0 artists (rules: min 1 Design mandated, max 9 total, $D+A \leq 3$; see [team-formation.pdf](#)). With 0 artists, the designer takes the **D11 conditional** as a real graded Design goal each milestone (M1 Art Proof of Concept / M2 Art Implementation Quality) — the last Design goal each milestone. *Note:* this is the **solo-designer + 0-artist** case the design pool flags — one designer now carries the full 4-goals-per-milestone Design + UI/UX load *and* the D11 art surface, which is a heavy load for a UX-heavy social app; a 2nd designer would relieve it if the team can, and the instructor should confirm which conditional bends if the load is too tight.

Track goals — suggested Design / UI-UX / Art / TEAM picks for this project; deliverables & quality ladders live in the track pool documents.

- **Design M1:**

- M1D1 ★ Features Design Document (the submit → validate → discover → plan-day-out loop, the trust model, and the "SG got what" positioning)
- M1D2 Core-Loop Interactive Prototype (Figma flow: nearby map → discover/feed → events filter → day-out itinerary → submit a spot)
- M1D11 Art Proof of Concept (conditional — no artist: map-marker/category-icon/trust-badge set, UI style guide)

- **Design M2:**

- M2D1 ★ Features Design Iteration & Showcase Readiness
- M2D11 Art Implementation Quality (conditional — no artist: UI art/icons implemented to standard)

- **UI/UX M1** (graded by the UI/UX instructor):

- M1U1 ★ UI Standards & Wireframe Baseline — map, feed, events, itinerary, profile and submission screens.

- **UI/UX M2:**

- M2U1 ★ Functional UI & Usability — the required screens navigable unaided on the mobile target.
- M2U2 ★ UX Validation & Final Polished UI — validated, polished map, feed and events browse.

- **Art:** 0 artists — the art surface is covered by the D11 conditionals, taken as Design goals by the single designer (see Design M1/M2).

- **TEAM M1:**

- M1TA1 ★ Overall Audio Experience — light UI/notification SFX and optional ambience from the week-5 needs list, present & working in the prototype (submit confirm, validated, nearby alert), suitable and correctly triggered. Audio is not load-bearing here — a lecturer scope waiver may apply.
- M1TA2 ★ Audio Engine Functionality — data-driven audio pipeline (add/replace files without recompiling) + reliable multi-channel playback for overlapping UI SFX.

- **TEAM M2:**

- M2TA1 ★ Overall Audio Experience — the standard floor tailored down for a utility app (subtle UI/notification SFX + optional ambience in the showcase build).
- M2TA2 ★ Audio Engine Functionality — audio properties de/serialised and the full playback functionality the build needs (multi-channel mixing for stacked notifications).
- M2TP1 ★ Final Presentation — 7 min in front of class + instructors: live custom-tools showcase (the web moderation/content editor), prototype demo, team post-mortem (wk14)